Flux-RL: Throughput-Aware Workload Scheduling for RLHF Post-Training

Anonymous Author(s)

Abstract

Reinforcement Learning from Human Feedback (RLHF) with long-context Large Language Models (LLMs) presents a distinct systems challenge: the training bottleneck shifts from the compute-bound prefill phase to the memory-bound and stochastic decode phase. As a result, standard length-based curricula fail in RLHF, either causing memory fragmentation under static batching or degrading learning when contexts are truncated too aggressively.

We introduce Flux-RL, a throughput-aware scheduling framework that co-schedules context length, generation bounds, and batch size for long-context RLHF. Flux-RL identifies and exploits transient memory slack arising from stochastic generation during early training, dynamically improving packing efficiency without compromising semantic correctness. We present a cost model showing how co-scheduling controls memory fragmentation and stabilizes execution under synchronized RLHF. Evaluated on a 32B-parameter model across 32 NPUs, Flux-RL substantially improves early-stage packing efficiency and token throughput, while converging to baseline efficiency once context and generation lengths plateau. As well as improving system throughput by (2.5×).

CCS Concepts

• **Computing methodologies** → **Artificial intelligence**; **Distributed algorithms**; • **Computer systems organization** → *Distributed architectures.*

Keywords

RLHF, Large Language Models, Workload Scheduling, NPU Memory Optimization, High Performance Computing

ACM Reference Format:

Anonymous Author(s). 2018. Flux-RL: Throughput-Aware Workload Scheduling for RLHF Post-Training. In *Proceedings of July 13–16, 2026 (HPDC '26).* ACM, New York, NY, USA, 10 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Large Language Models (LLMs) have evolved from static text generators into versatile agents capable of complex reasoning, tool use, and long-context synthesis [1, 14]. As these capabilities grow, the focus of model training has shifted from the initial pre-training phase to Post-Training alignment, specifically Reinforcement Learning from Human Feedback (RLHF) [2, 7].

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

HPDC '26,

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/XXXXXXX.XXXXXXX

Over the last few years, the operational envelope for these models has expanded by orders of magnitude. While early deployments operated in the 1–4k token regime, modern systems target hundreds of thousands to million-scale token windows [10]. This trend is especially pronounced in RLHF pipelines, where prompts frequently include multi-turn transcripts, tool traces, and retrieved documents. Simultaneously, response requirements have grown to support long-form "Chain of Thought" reasoning [12], increasing both the mean decode workload and its variance.

This expansion in context length is not merely a modeling choice but a systemic shift in how language models are deployed. Early transformer architectures were constrained by quadratic attention costs and modest memory budgets, making long-context processing impractical outside specialized tasks. Recent architectural, algorithmic, and hardware advances including optimized attention kernels, paged KV caches, and high-bandwidth memory, have turned long-context inference into a first-class workload. As a result, modern training pipelines must now treat context length not as a static hyperparameter, but as a dynamic systems variable that interacts directly with memory capacity, bandwidth, and synchronization behavior.

Scaling RLHF to these long contexts introduces bottlenecks distinct from pre-training. Standard pre-training is *compute-bound*, dominated by the quadratic cost of self-attention during the parallel *prefill* phase [4]. Consequently, curriculum strategies like SkyLadder [16] successfully accelerate pre-training by strictly modulating context length (L) to reduce FLOPs.

In contrast, RLHF is governed by the *autoregressive decode* phase. This phase is memory-bandwidth bound, with an arithmetic intensity orders of magnitude lower than prefill [6]. Furthermore, the aggregate decode cost grows linearly in context length but quadratically in total generated length (G). As a result, simply reducing the prompt length L—while leaving the generation target G fixed—yields negligible wall-clock speedups.

Worse, the naive application of length-based curricula to RLHF creates a resource utilization paradox. By reducing input sequence lengths without dynamically adjusting batch dimensions, the system fails to saturate the NPU's High Bandwidth Memory (HBM). This results in "fragmented" parallelism, where powerful accelerators sit idle. In distributed settings, this inefficiency is exacerbated by the stochastic nature of generation: the variance in response lengths leads to straggler effects, stalling the synchronization required for policy updates: Proximal Policy Optimization(PPO) [11], Group Relative Policy Optimization(GRPO) [12] or Direct Preference Optimizer(DPO) [8].

To address these challenges, we introduce **Flux-RL**, a throughput-aware scheduling framework designed for the memory-bound regime

of RLHF. Unlike static strategies, Flux-RL treats the training workload as elastic. We propose a co-scheduling algorithm that dynamically adjusts Context Length (L_t), Generation Bounds (G_t), and Batch Size (B_t) in unison.

Our key insight is that while we cannot change the memory-bound nature of decoding, we can linearize its cost. By enforcing a generation envelope $G_t \propto L_t$ and utilizing the freed HBM capacity to pack more sequences (Compute-Elastic Batching), Flux-RL maintains constant token-in-flight occupancy throughout the training run.

This paper makes the following contributions:

- **Systems Analysis of RLHF Bottlenecks:** We formalize a cost model for RL post-training that exposes the *Decode Dominance Threshold*, showing why pre-training optimizations based solely on context-length reduction fail in memory-bound RLHF workloads.
- **Throughput-Aware Scheduling Framework:** We introduce Flux-RL, a co-scheduling controller that jointly adjusts context length, generation bounds, and batch size. Flux-RL decouples inference and optimization batch dimensions, enabling dynamic exploitation of memory elasticity under synchronized RLHF execution.
- **packing efficiency and Utilization Gains:** Through large-scale experiments on a 32B-parameter model across 32 Ascend 910B3 NPUs, we show that Flux-RL increases peak token throughput by up to 2.5× and improves memory packing efficiency by up to 1.4× compared to fixed-batch baselines, without degrading convergence when semantic constraints are respected.
- **Semantic Boundary in RLHF Scheduling:** We identify and empirically validate a *Semantic Boundary* for RLHF: aggressive scheduling below a task-dependent minimum context length suppresses reward signals and stalls learning. Flux-RL makes this boundary explicit and tunable, enabling principled trade-offs between utilization and learning stability.

2 System Analysis & Motivation

To understand the systems challenges of RLHF, we first formalize the computational lifecycle of a training iteration. We then analyze why standard length-based curricula (like SkyLadder) fail in this regime, and finally present a theoretical cost model that explains these failures and guides our solution.

2.1 Lifecycle of an RLHF Iteration

We consider a distributed RLHF training pipeline where a policy π_θ is fine-tuned using PPO or GRPO. The workload for iteration t is defined by the tuple (L_t, G_t, B_t), where L_t is the prompt context length, G_t is the maximum generation length, and B_t is the micro-batch size per device.

Each iteration comprises three distinct phases:

(1) **Prefill (Compute-Bound):** The model processes the prompt L_t. The computational complexity is dominated by the attention mechanism, scaling as $O(B_t L_t^2)$.
(2) **Decode (Memory-Bound):** The model autoregressively generates a response of length $\tilde{g} \leq G_t$. Each token generation step requires reading the entire KV-cache and model weights from HBM. The complexity scales as $O(B_t(L_t G_t + G_t^2))$.
(3) **Update (Mixed):** The model computes gradients over the full sequence $L_t + \tilde{g}$.

Unlike pre-training, where the prefill phase dominates, RLHF is bottlenecked by the Decode phase. We formally define the *Decode Dominance Threshold* as the point where the decode cost exceeds the prefill cost.

Proposition 2.1 (Decode Dominance). *Given model-specific constants A (prefill), B_1 (cross-attention), and C (decode), the decode phase dominates the iteration time when the generation length G satisfies:*

$$G \geq L \cdot \frac{-B_1 + \sqrt{B_1^2 + 4CA}}{2C} \approx 0.732L \tag{1}$$

For standard KV-caching implementations ($A = B_1 = 1, C = 0.5$), this threshold is linear. Thus, for any meaningful long-context generation ($G \approx L$), RLHF is strictly decode-bound.

2.2 Failure Mode: SkyLadder

To illustrate the impact of this decode dominance, we evaluate SkyLadder [16], a curriculum learning strategy originally developed for pre-training. SkyLadder progressively increases context length L_t to reduce FLOPs.

Empirical Failure. As shown in Figure 1, naive application of SkyLadder to post-training fails. Despite the prompt length curriculum (Fig 1c), the total training time increases by **31%** compared to a fixed-context baseline for a fixed-token budget training. This counter-intuitive result—where processing *fewer* tokens takes *more* time—stems from a fundamental mismatch between length-based scheduling and the mechanics of distributed RLHF:

1. The Straggler Effect. The primary bottleneck in distributed RLHF is the Synchronization Barrier. In pre-training, batches are dense and deterministic; the compute cost is uniform across ranks. In RLHF, the pipeline is gated by the autoregressive generation phase, which runs for $K = \max_{i \in B}(\tilde{g}_i)$ steps, where $\tilde{g}_i$ is the stochastic length of the i-th sequence.Crucially, SkyLadder reduces the prompt length L_t, but leaves the generation limit G_t constant.

- Pre-training is Deterministic: $Cost \propto L^2$. Reducing L yields quadratic savings.
- RLHF is Stochastic: $Cost \propto \max(\tilde{g})$. Even if the prompt is short ($L = 1024$), the model may still generate a long "Chain-of-Thought" response ($G = 2048$).

Because the entire distributed group must wait for the single slowest generation to complete before proceeding to the PPO update, the step latency is effectively lower-bounded by the tail latency of the generation distribution. Reducing L_t does nothing to lower this floor.

2. Memory Fragmentation. The second inefficiency arises from the rigidity of standard training frameworks (e.g., DeepSpeed, Megatron-LM), which fix the Global Batch Size (GBS) to ensure hyperparameter stability.Let M_{avail} be the device memory and $M_{req}(t) = B_{static} \cdot (L_t + G_{static})$ be the memory required at iteration t.In the

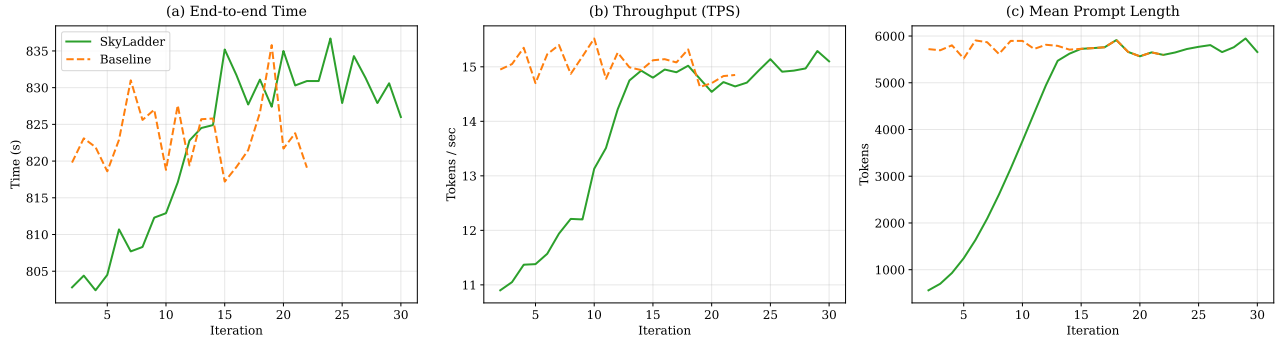

Figure 1: The Failure of Naive Scheduling. (a) Naive SkyLadder (Brown) results in higher step latency than Baseline (Orange) after the ramp-up. (b) Throughput is low because reducing L_t without increasing Batch Size B_t leaves HBM fragmented. (c) Prompt length progression.

early phases of SkyLadder, where $L_t \ll L_{\max}$, we observe:

$$M_{req}(t) \ll M_{avail} \tag{2}$$

This creates massive Memory Slack—HBM pages that are powered, clocked, but empty.In pre-training, this is acceptable because the workload is compute-bound; the GPU cores are saturated even with smaller memory footprints. In RLHF, however, the decode phase is memory-bandwidth bound with low arithmetic intensity. To saturate the HBM bandwidth, one must maximize the number of active sequences (batch size) to hide the memory access latency. By keeping B_t static while L_t shrinks, the system fails to hide this latency, causing the NPU compute units to stall waiting for memory fetches. The "saved" memory from shorter prompts is effectively wasted, leading to the severe throughput degradation seen in Figure 1(b)

3 Theoretical Analysis

The empirical failure of SkyLadder highlights the need for a rigorous cost model. We formalize the interaction between memory capacity, stochastic generation, and wall-clock latency to derive the design principles for Flux-RL.

3.1 System Model

Fix a device memory token cap $T_{\max} > 0$ and a constant gradient update overhead $T_{\text{upd}} \geq 0$. The scheduler must select a batch size B_t such that the memory footprint does not exceed the hardware limit in the worst case:

$$B_t = \left\lfloor \frac{T_{\max}}{L_t + G_t} \right\rfloor \tag{3}$$

While memory is provisioned for the worst case G_t, the *actual* compute cost is driven by the realized generation length $\tilde{g}_{t,i} \in [0, G_t]$, which is an i.i.d. random variable with mean μ_t and second moment ν_t.

3.2 The Variance Bottleneck

In RLHF, step time is driven by the realized generation length $\tilde{g}$, not just the allocated capacity.

Theorem 3.1 (Stochastic Step Variance). *Assume the per-sequence compute cost is quadratic $F(L, \tilde{g}) = AL^2 + B_1 L\tilde{g} + C\tilde{g}^2$. The expected step time under continuous batching is:*

$$\mathbb{E}[T_t] = \alpha B_t \left(AL_t^2 + B_1 L_t \mu_t + C\nu_t\right) + T_{\text{upd}} \tag{4}$$

Furthermore, the variance (jitter) of the step time is bounded by the square of the generation cap:

$$\text{Var}(T_t) \lesssim (\alpha B_t)^2 \cdot \left(B_1 L_t + 2CG_t\right)^2 \cdot \frac{G_t^2}{4} \tag{5}$$

Proof sketch. The bound follows by expanding the quadratic cost in $\tilde{g}$, applying Popoviciu's inequality $\text{Var}(\tilde{g}) \leq G_t^2/4$, and upper-bounding cross terms by their worst-case coefficients. Full derivation is provided in Appendix A.

Implication: This theorem explains the instability of aggressive scheduling (Green line in earlier plots). While reducing L_t allows for a larger B_t, keeping G_t loose results in high variance ($\propto G_t^2$). Flux-RL's co-scheduling of G_t is essential to minimize this jitter.

3.3 Throughput Optimality

We define the *Allocated Throughput* $S(B)$ as the theoretical tokens-per-second capacity of the hardware.

Theorem 3.2 (Optimality of Packing). *Assuming the per-token processing time $\phi(n)$ is non-increasing up to the memory limit $T_{\max}$, the system throughput is strictly maximized by saturating the memory constraint:*

$$B_t^* = \left\lfloor \frac{T_{\max}}{L_t + G_t} \right\rfloor \tag{6}$$

Implication: This justifies the "Compute-Elastic" controller. Any static batch size $B < B^*$ (like the Baseline's $B = 16$) leaves memory stranded ($\mathcal{L}_{pack} > 0$), resulting in suboptimal throughput. This argument is identical to throughput-optimal packing results in serving systems (e.g., vLLM), but applied here under synchronized RLHF constraints.

3.4 Packing Efficiency & Speedup

Allocated tokens are not always executed tokens. We define *Packing Efficiency* η_t as the ratio of useful work to allocated capacity:

$$\eta_t = \frac{n_t^{\text{exec}}}{n_t^{\text{alloc}}} = \frac{L_t + \mu_t}{L_t + G_t} \tag{7}$$

Since the actual generation length μ_t is strictly bounded by the allocation G_t, it follows that $\eta_t \in [0, 1]$. A static baseline reserves memory for the worst case ($G_t = G_{\max}$), leading to low efficiency ($\eta_t \ll 1$) when responses are short. Flux-RL tightens the allocation $G_t \approx \mu_t$, driving $\eta_t \to 1$.

Theorem 3.3 (Wall-Clock Speedup). *Let the baseline use fixed caps (L_0, G_0). The total wall-clock speedup of a schedule $\mathcal{S} = \{(L_t, G_t)\}$ is proportional to the improvement in cumulative packing efficiency:*

$$\frac{\mathbb{E}[T_{\text{tot}}]}{T \cdot \mathbb{E}[T_0]} \approx (1 - s_{\text{upd}}) \cdot \frac{\sum \eta_t^{-1}}{\sum \eta_0^{-1}} + s_{\text{upd}} \tag{8}$$

Proof sketch. Since step time decomposes into an update component and a token-dependent component, the expected total time is a weighted sum of inverse packing efficiencies. Summing over iterations yields the stated ratio. Details are provided in Appendix A.

Implication: Speedup is driven not just by shorter prompts but by maximizing η_t. Flux-RL achieves this by tightening G_t to match μ_t (the model's intrinsic length), driving $\eta_t \to 1$.

Proposition 3.4 (Dataset Velocity). *Let the dataset contain N prompts. The number of iterations τ required to complete one epoch is the first index such that $\sum_{t=0}^{\tau-1} B_t \geq N$. Since Flux-RL enforces $B_t^{Flux} \geq B^{Base}$ for all t, it follows strictly that $\tau_{Flux} < \tau_{Base}$.*

Implication. Flux-RL reduces time-to-data independently of optimization dynamics, making it strictly more sample-efficient under fixed wall-clock budgets.

4 Flux-RL Design

To overcome the decode dominance and memory fragmentation inherent in RLHF, Flux-RL implements a throughput-aware curriculum that treats NPU memory as an elastic resource by dynamically reallocating token capacity across stages.The framework consists of two main components: a *Workload Scheduler* that determines the global (L_t, G_t) envelope, and a *Compute-Elastic Controller* that decouples batch sizes across stages to maximize utilization.

4.1 Throughput-Aware Scheduling

Intuitively, Flux-RL reframes curriculum learning as a scheduling problem rather than a purely optimization-driven heuristic. Instead of shrinking sequences to reduce compute, we shrink them to free memory, then immediately reinvest that memory to increase parallelism. This shift is crucial in the decode-dominated regime, where per-token latency is fixed but throughput can still be increased via packing.

The Generation Envelope. In standard RLHF, the generation length is typically fixed to the maximum required response length (e.g., $G = 2048$) regardless of the prompt length. Flux-RL introduces a dynamic generation cap G_t that scales with the prompt length L_t:

$$G_t = \min\{\lceil \rho L_t \rceil, G_{max}, M - L_t\} \tag{9}$$

where ρ is the target decode-context ratio (typically ≈ 1.0). Empirically, we observe that for long-horizon reasoning tasks, the expected response length scales proportionally with prompt length, making $\rho \approx 1$ a stable operating point. G_{max} is the task-specific limit, and M is the model's absolute context window. By tying G_t to L_t, we ensure that early iteration (short context) also incur low decode costs, effectively lowering the Decode Dominance threshold derived in Section 2.

Cosine Context Ramp. We employ a cosine schedule to transition L_t from a minimum sufficient context L_{min} to the full target L_{max} over a fraction f of the total training steps T_{total}:

$$L_t = L_{min} + (L_{max} - L_{min}) \cdot \frac{1 - \cos(\pi \cdot \min(1, t/(f \cdot T_{total})))}{2} \tag{10}$$

This smooth progression prevents the "shock" of sudden computational load increases, stabilizing the PPO updates.

4.2 Compute-Elastic Batching

The most critical systems contribution of Flux-RL is the virtualization of the batch size. In a distributed RL pipeline, different stages consume data at different rates. A static global batch size B_{global} invariably bottlenecks the most memory-intensive stage (usually the joint forward pass), leaving the inference stage underutilized.

We implement **Decoupled Batching**, where the batch size $B_t^{(s)}$ is calculated independently for each stage $s \in \{Rollout, Update\}$ based on its specific token consumption c_s:

- Rollout Stage ($c = L_t$): Only processes prompts.
- Update Stage ($c = L_t + G_t$): Processes full sequences + gradients.

The controller dynamically computes the optimal batch size $B_t^{(s)}$ to saturate the device token limit T_{max}:

$$B_t^{(s)} = \text{granule}\left(\left\lfloor \frac{T_{max}}{c_s} \right\rfloor\right) \tag{11}$$

where granule$(\cdot)$ ensures alignment with hardware constraints (e.g., multiples of 8 for tensor cores).

Algorithm 1 Compute-Elastic Batching

Require: Scheduled (L_t, G_t); device token cap $T_{\max}$; stage set $\mathcal{S}$; granularity q (e.g., multiple-of-8); upper bounds $B_{\max}^{(s)}$.
Ensure: Per-stage batch sizes $\{B_t^{(s)}\}_{s \in \mathcal{S}}$.

```
for all s ∈ S do
    c_s ← StageTokens(s, L_t, G_t)   ▹ e.g., Rollout: c = L_t + G_t; Update: c = L_t + G_t; Prompt-only stages: c = L_t
    B̂ ← ⌊T_max / c_s⌋
    B_t^(s) ← q · ⌊B̂ / q⌋   ▹ Granularity rounding
    B_t^(s) ← min(B_t^(s), B_max^(s))
end for
return {B_t^(s)}_{s∈S}
```

This design mirrors virtualization techniques in distributed systems, where resources are overcommitted and multiplexed to hide latency. In Flux-RL, batch size becomes a virtual quantity: the inference engine sees a large effective batch, while the optimizer operates on a smaller, stable batch suitable for convergence.

Algorithm 2 Flux-RL Scheduler

Require: Iteration t; total iterations T; ramp fraction f; min/max context $(L_{\min}, L_{\max})$; model limit M; decode ratio ρ; cap $G_{\max}$; alignment granule m.
Ensure: Scheduled (L_t, G_t) broadcast to all ranks.

$u \leftarrow \min\left(1,\ \frac{t}{fT}\right)$ ▷ Normalized ramp progress
$r \leftarrow \frac{1-\cos(\pi u)}{2}$ ▷ Cosine ramp in $[0, 1]$
$L_t \leftarrow \textsc{Align}(L_{\min} + (L_{\max} - L_{\min}) \cdot r,\ m)$
$G_t \leftarrow \min(\lceil \rho L_t \rceil,\ G_{\max},\ M - L_t)$
Broadcast (L_t, G_t) to all ranks

4.3 Stochastic Resource Underutilization

In a real-world system, the generation length is probabilistic. The model stops generating upon emitting an EOS token or reaching the hard cap G_t. Let $\tilde{g}_i$ be the actual length of the i-th sequence in batch B_t, drawn from a distribution $P(\tilde{g}|L_t)$.

System efficiency is constrained by the *worst-case* allocation within a synchronous batch:

$$G_{effective} = \max_{i \in \{1..B_t\}} (\tilde{g}_i) \tag{12}$$

However, memory must be pre-allocated for the theoretical maximum G_t to prevent Out-Of-Memory (OOM) errors during execution. This creates *Execution Slack*—allocated but unused memory and compute cycles.

We define the **Packing Loss** $\mathcal{L}_{pack}(t)$ as the ratio of wasted capacity:

$$\mathcal{L}_{pack}(t) = 1 - \frac{\sum_{i=1}^{B_t}(L_t + \tilde{g}_i)}{B_t(L_t + G_t)} \tag{13}$$

When G_t is large and fixed (static baseline), the variance of $\tilde{g}$ leads to high Packing Loss. Flux-RL mitigates this by strictly bounding the variance envelope. By setting $G_t \propto L_t$ in early stages, we squeeze the probability distribution of $\tilde{g}$, forcing $\max(\tilde{g}_i) \to G_t$, thereby minimizing slack and maximizing the effective arithmetic intensity.

This formulation directly motivates the variance bounds derived in Section 3.2: reducing G_t reduces packing loss, hence stabilizing distributed synchronization. Flux-RL does not eliminate the stochastic nature observed in generation or optimization. Instead, it tightens the worst-case variance envelope of the decode workload, preventing step-time spikes and enabling stable throughput under stochastic generation.

4.4 Handling Stochastic Generation

To address the variance in actual generation lengths $\tilde{g}$, the controller utilizes a centralized state dictionary to synchronize the *effective* batch dimensions across ranks. Since vLLM utilizes continuous batching, it may process requests faster than the training loop. Flux-RL acts as a barrier, enforcing the calculated B_t as a hard constraint on the optimizer to ensure that gradient updates are computed over a consistent number of tokens, effectively mitigating the straggler effect caused by variance $\tilde{g}$. Flux-RL does not disable continuous batching inside the inference engine. Instead, it enforces synchronization at the optimizer boundary, ensuring that each update consumes a consistent token budget even when individual requests complete early.

5 Evaluation

We evaluate Flux-RL to answer three key questions:

(1) **Per-step Efficiency:** Does co-scheduling reduce wall-clock time per optimizer step under identical synchronization?
(2) **Utilization:** Does compute-elastic batching effectively saturate hardware throughput?
(3) **Quality:** Does the curriculum maintain model convergence compared to full-context baselines?

5.1 Experimental Setup

Experiments were conducted on a cluster of 32 Huawei Ascend 910B3 NPUs. We fine-tune a Qwen3-32B model [1] using GRPO. We compare three distinct configurations:

- **Baseline (Orange):** We ran a fixed context $L = 8192$ with not constraints on decode, global batch size (GBS) = 16 and dispatch size = 32. Note that attempts to run GBS= 32 resulted in immediate OOM errors.
- **Flux-RL Conservative (Blue):** Starts at $L_{min}, G_{min} = 4096$ to ensure full coverage of prompt semantics. Scales batch size dynamically between $B \in [8, 16]$.
- **Flux-RL Aggressive (Green):** Starts at $L_{min}, G_{min} = 1024$ to maximize throughput. Scales batch size dynamically between $B \in [32, 64]$.

All runs are trained on the same dataset and use the same synchronizer, optimizer and

5.1.1 Comparison Protocol and Fairness. Flux-RL is a systems-level scheduling framework for RLHF workloads. Our evaluation therefore focuses on per-iteration systems efficiency. An exact fixed-token comparison is infeasible for two reasons:

- the baseline run uses checkpoint initialization, starting from a converged policy, whereas Flux-RL exposes the full learning trajectory regardless of checkpointing.
- Flux-RL dynamically varies batch size and completes dataset traversal earlier, leading to different token consumption profiles.

To ensure a fair comparison, we adopt optimizer-step–aligned metrics, measuring:

- wall-clock time per iteration,
- throughput (tokens/s),
- hardware utilization,
- dataset velocity (iterations to exhaust a fixed dataset)

All methods are evaluated under identical synchronization, optimizer, and reward computation constraints.

5.2 Throughput vs. Semantics

Figure 3 illustrates the critical boundary between systems efficiency and model learning.

The Throughput Ceiling (Green Line). The Aggressive configuration ($L_{start} = 1024$) demonstrates the raw potential of the Flux-RL engine. The scheduler achieves an end-to-end throughput of $\approx 150 - 180$ tokens/s, a $2.5\times - 3\times$ speedup over the Baseline (≈60

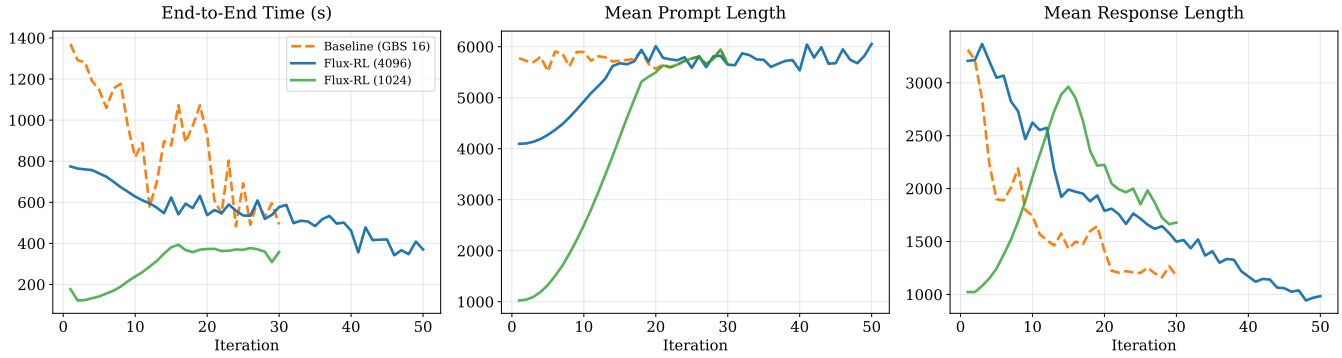

Figure 2: Latency and Workload Evolution. (Left) Flux-RL drastically reduces step time in the first 20 iterations. (Center/Right) The Prompt Length ramps up according to the curriculum. Notably, the Response Length (Right) drops naturally for all runs; Flux-RL capitalizes on this by increasing batch size as response lengths decrease, whereas the Baseline leaves this capacity unused.

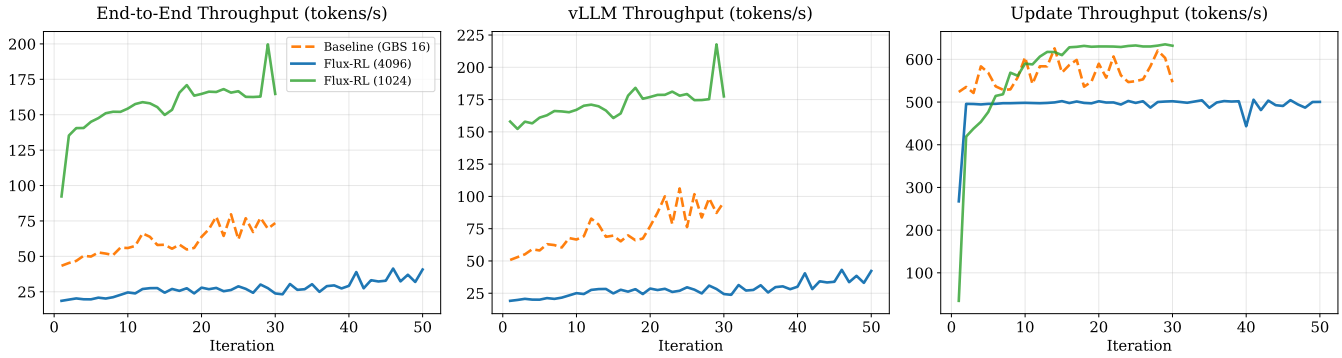

Figure 3: Throughput Analysis. The Aggressive Schedule (Green, 1024) achieves massive throughput gains ($\sim 2.5\times$ baseline) by maximizing batch size. The Conservative Schedule (Blue, 4096) achieves a lower token/s than Baseline (Orange), the reason being Baseline was run at twice the GBS and dispatch sizes as Flux-RL.

TPS). A similar pattern is noticed with vLLM throughput.(3) Moreover, this throughput advantage does not collapse as context and generation lengths increase. After iterations ≈ 20, when L_t and G_t approach their maximum values, both end-to-end and vLLM throughput stabilize, confirming that Flux-RL converts early memory slack into persistent utilization gains rather than temporary warm-up effects. This confirms that *Compute-Elastic Batching* successfully virtualizes memory, converting freed HBM into massive batch sizes ($B = 64$) during the rollout phase.

The Goldilocks Zone (Blue Line). The Conservative configuration ($L_{start} = 4096$) represents the optimal operating point for this task. While it sacrifices throughput compared to the Aggressive run, it maintains a context window large enough to capture the full prompt logic. It consistently runs faster (~400s/step) than the Baseline (~1300s/step) (2) during the early phases while matching the Baseline's convergence quality. Regarding throughput, Flux-RL's consistently lower throughput compared to baseline can be explained by: firstly, baseline was running $2\times$ the batch sizes for each stage as Flux-RL's maximum batch size B_{max}; secondly, baseline $(131,072)$ was processing $2\times$ the tokens per step compared to Flux-RL $(65,536)$. Regarding update throughput, we notice that the Conservative run achieves an almost perfectly constant throughput of $\sim$ 500 tps. Though the Aggressive run has an increasing period for update throughput, it also achieves a nearly constant throughput of $\sim$ 620 tps after iteration 16.

5.3 The Truncation Trap in RLHF

Unlike pre-training, RLHF imposes a semantic constraint: rollouts must be sufficiently complete to produce meaningful reward signals.

As shown in 4, the Aggressive configuration exhibits near-zero GRPO improvements for the first ≈ 15 iterations. This behaviour is caused by *Semantic Truncation*: the generation envelope ($G_t \propto L_t \propto 1024$) feeds an incomplete prompt to the model, hence preventing the model from completing reasoning trajectories. This effectively suppresses the reward signal.

In contract, both baseline and Conservative Flux-RL runs allow full semantic rollouts fomr iteration 0 and exhibit stable convergence. This reveals a hard lower bound for RLHF scheduling:

$$L_{min} \geq L_{semantic} \tag{14}$$

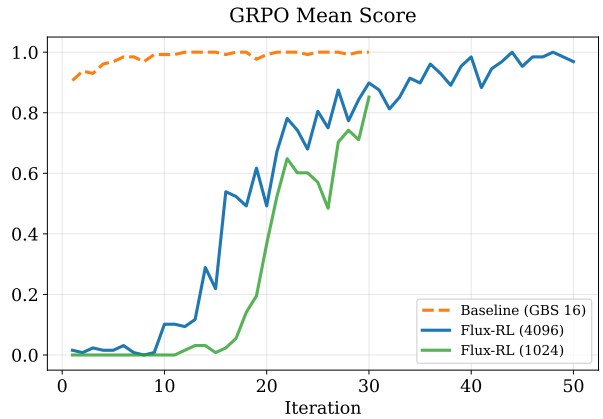

Figure 4: Convergence Stability (GRPO Score). The Aggressive run (Green) fails to learn initially due to *Semantic Truncation*. The Conservative run (Blue) avoids this trap, matching the Baseline's (Orange) convergence.

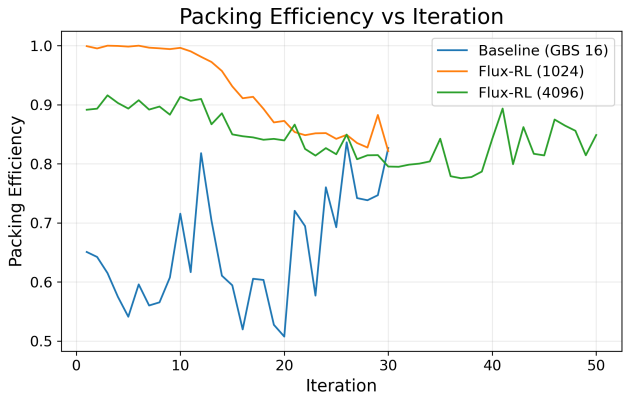

Figure 5: Measured packing efficiency across training iterations. Flux-RL consistently attains higher packing efficiency than the fixed-batch Baseline, indicating reduced memory fragmentation under dynamic batching. packing efficiency converges as context and generation lengths approach their maximum values.

where $L_{semantic}$ is the minimum context required to express task-level reasoning. Cruicially, Flux-RL makes this boundary explicit and tunable, whereas static baselines implicitly assume it.

This phenomenon highlights a key distinction between throughput-optimal scheduling and learning-optimal scheduling. Unlike inference or pre-training workloads, RLHF requires that each rollout produce semantically complete trajectories to generate meaningful reward signals. The following constraint must be satisfied: L_{min} must be lower-bounded by the dataset's *minimum prompt length*, so that it has enough context to allow the model to reason. Scheduling policies that aggressively optimize hardware utilization without respecting this constraint can stall learning as seen with the Aggressive run.

5.4 Dataset Completion & Packing Efficiency

Dataset Completion. Because Flux-RL increases trains at large batch sizes during the early phases ($B_t \gg B_{\text{baseline}}$), it processes more samples per iteration under identical synchronization constraints.

As a result, Flux-RL exhausts the fixed dataset in fewer optimizer steps than the baseline. This improvement is orthogonal to per-step throughput and reflects a distinct systems advantage: faster data traversal under synchronized RLHF execution.

Because Flux-RL processes more samples iteration during early phases, it traverses a fixed dataset in fewer optimizer steps. This creates additional flexibility in how a fixed wall-clock budget may be allocated. We do not claim end-to-end training speedup under a fixed token budget due to lack of data.

packing efficiency. A central objective of Flux-RL is not merely to increase batch size, but to improve how effectively device memory is translated into useful computation. To quantify this effect, we measure packing efficiency as defined in section 3.4. Figure 5 reports packing efficiency across training iterations for the Baseline and both Flux-RL configurations. Several observations emerge.

First, the Baseline exhibits consistently low packing efficiency in early training, fluctuating between approximately 0.55–0.7. This inefficiency arises because the baseline allocates memory for worst-case generation lengths while actual response lengths are initially much shorter. Since batch size is fixed, this slack cannot be reclaimed, leading to under-utilization of HBM during the most throughput-critical phase of training.

Second, Flux-RL significantly improves packing efficiency during the curriculum phase. The Aggressive configuration (1024) starts near perfect efficiency and gradually decreases as generation lengths increase, reflecting tighter coupling between the generation envelope and realized execution. The Conservative configuration (4096) maintains packing efficiency in the range 0.85–0.9 throughout early training, consistently outperforming the baseline. Importantly, these improvements arise not from predicting response length, but from dynamically resizing batch dimensions to absorb freed memory capacity.Section.

Third, packing efficiency converges across all methods once the curriculum plateaus. As L_t and G_t approach their maximum values, the stochastic slack inherent to generation diminishes and all runs operate near a similar efficiency regime. This convergence indicates that Flux-RL does not rely on transient warm-up artifacts; instead, it improves memory utilization precisely when slack exists and naturally relinquishes this advantage as the workload becomes fully saturated.

Overall, these results support the core systems insight of Flux-RL: memory inefficiency in RLHF is a transient but exploitable phenomenon. While we do not claim end-to-end speedup under a fixed token budget, improved packing efficiency represents a strictly stronger systems metric—it measures how effectively each allocated unit of HBM is converted into executed tokens under synchronized RLHF constraints.

6 Limitations and Future Work

6.1 Limitations

Fixed-token end-to-end comparisons. Our evaluation focuses on per-iteration systems efficiency (step latency, throughput, and utilization) rather than fixed-token or fixed-epoch end-to-end training time. An exact token-budget–matched comparison is infeasible in our current setup because the baseline run relies on checkpoint initialization and static batching, whereas Flux-RL exposes the full learning trajectory and dynamically varies batch size and dataset traversal rate. As a result, total tokens consumed and dataset coverage differ across runs. We therefore treat learning quality as a constraint (no regression) rather than a primary comparison axis.

Dataset-progress instrumentation. While Flux-RL demonstrably increases samples processed per iteration in early phases, we do not directly report baseline time-to-epoch due to missing dataset-consumption logs. Consequently, our dataset-velocity analysis is based on per-step batch expansion rather than explicit epoch completion time. Future experiments will instrument prompt- and token-level dataset counters across all baselines to enable direct time-to-epoch comparisons.

Batch-size quantization. Practical accelerators impose discrete batch-size constraints (e.g., powers of two or fixed granularities). Flux-RL therefore realizes a piecewise-constant batching policy rather than a continuous optimum. Although this quantization slightly reduces theoretical packing optimality, empirical results show that even coarse batch levels (e.g., 8, 16, 32, 64) capture most of the available memory slack, and throughput gains remain stable once the maximum batch size is reached.

Task- and dataset-specific semantic boundaries. The minimum viable context length (L_{semantic}) required to preserve reward signal is task-dependent. While Flux-RL exposes this boundary explicitly, selecting $L_{\min}$ still requires empirical validation. Aggressive schedules may therefore fail on tasks with long reasoning chains or heavy tool usage.

6.2 Future Works

Token-budget–matched evaluations. A natural next step is to perform fixed-token and fixed-epoch comparisons by instrumenting cumulative token and prompt counters for all baselines. This would enable precise end-to-end training time comparisons and quantify the dataset-completion advantage suggested by our current results.

Adaptive generation envelopes. Flux-RL currently enforces $G_t \propto L_t$ using a fixed decode ratio. Future work could adapt G_t online using observed response-length distributions or uncertainty estimates, further reducing packing loss while respecting semantic constraints.

Integration with learning-aware signals. While Flux-RL deliberately separates systems scheduling from learning dynamics, incorporating lightweight learning signals (e.g., reward variance or KL stability) could enable adaptive curricula that balance throughput and convergence more tightly.

Generalization beyond RLHF. The core idea of compute-elastic batching applies to any workload combining memory-bound inference with synchronized training, including agentic training loops, tool-augmented models, and multi-modal RL. Evaluating Flux-RL in these settings is a plausible direction for future research.

7 Related Work

7.1 Efficient Large Scale Training

The training of Large Language Models (LLMs) has been scaled to thousands of GPUs through 3D parallelism: Data Parallelism (DP), Tensor Parallelism (TP) [13], and Pipeline Parallelism (PP) [5]. Systems like DeepSpeed ZeRO [9] further optimize memory by partitioning optimizer states. While these methods solve the memory constraints of *compute-bound* pre-training, they are designed for static computational graphs and fixed sequence lengths. They do not address the dynamic, memory-bound fragmentation introduced by the stochastic decoding phase of RLHF.

7.2 Memory-Efficient Inference

Optimizing the decoding phase has been a primary focus for inference serving systems. PagedAttention (vLLM) [6] mitigates memory fragmentation by managing Key-Value (KV) caches as non-contiguous pages, akin to OS virtual memory. Orca [15] introduced iteration-level scheduling to handle varying request lengths. FlashAttention [4] optimizes the attention kernel itself to minimize HBM access. Flux-RL leverages these primitives but extends them to the training loop. Unlike pure inference systems, Flux-RL must co-schedule the decoding batch size with the gradient update batch size, requiring a unified view of memory that serving systems lack.

7.3 Curriculum Learning in LLMs

Curriculum learning [3] accelerates training by ordering data from easy to hard. In the context of LLMs, length-based scheduling—starting with short sequences and ramping up—has proven effective for pre-training efficiency. SkyLadder [16] formalizes this for pre-training, showing significant FLOPs reduction. However, our work demonstrates that directly transferring this paradigm to RLHF leads to the "Truncation Trap" (Section 3), where aggressive length reduction destroys the semantic validity of reasoning chains. Flux-RL represents the first curriculum approach specifically designed for the semantic and systems constraints of post-training.

8 Conclusion

RLHF efficiency is governed by memory occupancy and variance control, not FLOPs. We demonstrated that static batching strategies leave NPUs underutilized during the decode phase. However, we also identified a critical semantic boundary: aggressive length scheduling (SkyLadder) can destroy model convergence by truncating reasoning chains before they yield valid reward signals.

Flux-RL navigates this trade-off by decoupling inference batch sizes from optimization batch sizes. By respecting the model's intrinsic reasoning length while aggressively packing memory during rollout, Flux-RL achieves the Goldilocks outcome: it matches the convergence quality of full-context training while delivering the throughput of aggressive systems optimizations. As LLMs continue

to scale, such semantic-aware systems scheduling will be essential for efficient post-training.

References

[1] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Ge, Y. Gao, et al. 2023. Qwen Technical Report. arXiv:2309.16609 [cs.CL] https://arxiv.org/abs/2309.16609

[2] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, et al. 2022. Constitutional AI: Harmlessness from AI Feedback. arXiv:2212.08073 [cs.CL] https://arxiv.org/abs/2212.08073

[3] Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. 2009. Curriculum Learning. In *Proceedings of the 26th Annual International Conference on Machine Learning* (Montreal, Quebec, Canada) *(ICML '09)*. ACM, New York, NY, USA, 41–48. doi:10.1145/1553374.1553380

[4] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh (Eds.), Vol. 35. Curran Associates, Inc., 16344–16359.

[5] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V. Le, Yonghui Wu, and Zhifeng Chen. 2019. GPipe: Efficient Training of Giant Neural Networks Using Pipeline Parallelism. In *Advances in Neural Information Processing Systems*, H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché Buc, E. Fox, and R. Garnett (Eds.), Vol. 32. Curran Associates, Inc. https://proceedings.neurips.cc/paper/2019/file/093f65e080a295f8076b1c5722a46aa2-Paper.pdf

[6] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Ion Stoica, and Hao Zhang. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles* (Koblenz, Germany) *(SOSP '23)*. ACM, New York, NY, USA, 611–626. doi:10.1145/3600006.3613165

[7] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. Training Language Models to Follow Instructions with Human Feedback. In *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh (Eds.), Vol. 35. Curran Associates, Inc., 27730–27744. https://proceedings.neurips.cc/paper_files/paper/2022/hash/b1efde53be364a73914f58805a001731-Abstract-Conference.html

[8] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D. Manning, Stefano Ermon, and Chelsea Finn. 2023. Direct Preference Optimization: Your Language Model is Secretly a Reward Model. In *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (Eds.), Vol. 36. Curran Associates, Inc., 53702–53723. https://proceedings.neurips.cc/paper_files/paper/2023/hash/a85b405ed65c6477a4fe8302b5e06ce7-Abstract-Conference.html

[9] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. 2020. ZeRO: Memory Optimizations Toward Training Trillion Parameter Models. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis* (Atlanta, Georgia) *(SC '20)*. IEEE Press, Article 20, 16 pages. doi:10.1109/SC41405.2020.00024

[10] Machel Reid, Nikolay Savinov, Denis Teplyashin, Dmitry Lepikhin, Timothy Lillicrap, Jean-Baptiste Alayrac, Radu Soricut, Angeliki Lazaridou, Orhan Firat, Julian Schrittwieser, et al. 2024. Gemini 1.5: Unlocking Multimodal Understanding Across Millions of Tokens of Context. arXiv:2403.05530 [cs.CL] https://arxiv.org/abs/2403.05530

[11] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal Policy Optimization Algorithms. arXiv:1707.06347 [cs.LG] https://arxiv.org/abs/1707.06347

[12] Zhihong Shao, Peiyi Wang, Qiuju Zhu, Runxin Xu, Junxiao Song, Mingchuan Xiao, Y. K. Li, Xiaohan Zhang, and Yajie Yang. 2024. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. arXiv:2402.03300 [cs.CL] https://arxiv.org/abs/2402.03300

[13] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism. arXiv:1909.08053 [cs.CL] https://arxiv.org/abs/1909.08053

[14] Wenli Yang, Lilian Some, Michael Bain, and Byeong Kang. 2025. A Comprehensive Survey on Integrating Large Language Models with Knowledge-Based Methods. *Knowledge-Based Systems* 313 (2025), 113503. doi:10.1016/j.knosys.2025.113503

[15] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation* (Carlsbad, CA, USA) *(OSDI '22)*. USENIX Association, Berkeley, CA, USA, 521–538. https://www.usenix.org/conference/osdi22/presentation/yu

[16] Tongyao Zhu, Qian Liu, Haonan Wang, Shiqi Chen, Xiangming Gu, Tianyu Pang, and Min-Yen Kan. 2025. SkyLadder: Better and Faster Pretraining via Context Window Scheduling. arXiv:2503.15450 [cs.LG] https://arxiv.org/abs/2503.15450 Accepted to NeurIPS 2025.

A Proofs

A.1 Proof of Theorem 3.1 (Stochastic Step Variance)

Theorem 3.1 (restated). Fix an iteration t. Let the scheduled cap be $G_t > 0$, and let the realized generation lengths $\tilde{g}_{t,1}, \ldots, \tilde{g}_{t,B_t}$ be i.i.d. random variables supported on $[0, G_t]$. Assume the per-sequence compute cost takes the quadratic form

$$F(L_t, \tilde{g}) = AL_t^2 + B_1 L_t \tilde{g} + C\tilde{g}^2 \quad \text{with} \quad A, B_1, C \geq 0,$$

and the step time under (iteration-level) continuous batching can be written as

$$T_t \;=\; \alpha B_t \,\overline{F}_t \;+\; T_{\text{upd}}, \qquad \overline{F}_t := \frac{1}{B_t} \sum_{i=1}^{B_t} F(L_t, \tilde{g}_{t,i}),$$

where $\alpha > 0$ is a device-dependent constant and T_{upd} is deterministic overhead. Then:

$$\mathbb{E}[T_t] = \alpha B_t \big(AL_t^2 + B_1 L_t \mu_t + C\nu_t\big) + T_{\text{upd}}, \tag{15}$$

$$\text{Var}(T_t) \leq (\alpha B_t)^2 \cdot \text{Var}\big(F(L_t, \tilde{g})\big), \tag{16}$$

where $\mu_t := \mathbb{E}[\tilde{g}]$ and $\nu_t := \mathbb{E}[\tilde{g}^2]$. Moreover, a distribution-free bound holds:

$$\text{Var}(T_t) \;\leq\; (\alpha B_t)^2 \cdot \Big(B_1 L_t + 2CG_t\Big)^2 \cdot \frac{G_t^2}{4}. \tag{17}$$

Proof. **Step 1: expectation.** By linearity of expectation,

$$\mathbb{E}[\overline{F}_t] = \mathbb{E}\left[\frac{1}{B_t} \sum_{i=1}^{B_t} F(L_t, \tilde{g}_{t,i})\right] = \mathbb{E}[F(L_t, \tilde{g})],$$

where $\tilde{g}$ denotes a generic copy of $\tilde{g}_{t,i}$. Expanding F and using $\mu_t = \mathbb{E}[\tilde{g}]$, $\nu_t = \mathbb{E}[\tilde{g}^2]$ gives

$$\mathbb{E}[F(L_t, \tilde{g})] = AL_t^2 + B_1 L_t \mu_t + C\nu_t.$$

Plugging into $T_t = \alpha B_t \overline{F}_t + T_{\text{upd}}$ yields (15).

Step 2: variance reduction to a single-sequence term. Since T_{upd} is deterministic,

$$\text{Var}(T_t) = \text{Var}(\alpha B_t \,\overline{F}_t) = (\alpha B_t)^2 \,\text{Var}(\overline{F}_t).$$

Because $\tilde{g}_{t,1}, \ldots, \tilde{g}_{t,B_t}$ are i.i.d., the random variables $F(L_t, \tilde{g}_{t,i})$ are i.i.d., hence

$$\text{Var}(\overline{F}_t) = \text{Var}\left(\frac{1}{B_t} \sum_{i=1}^{B_t} F(L_t, \tilde{g}_{t,i})\right) = \frac{1}{B_t} \,\text{Var}\big(F(L_t, \tilde{g})\big).$$

Combining gives the exact identity

$$\text{Var}(T_t) = (\alpha B_t)^2 \cdot \frac{1}{B_t} \,\text{Var}(F(L_t, \tilde{g})) = \alpha^2 B_t \,\text{Var}(F(L_t, \tilde{g})).$$

In particular, the weaker inequality (16) holds.

Step 3: distribution-free bound via Lipschitz continuity and Popoviciu. Define $h(g) := F(L_t, g)$ on $[0, G_t]$. Then $h'(g) = B_1 L_t + 2Cg$, so for all $g \in [0, G_t]$,

$$|h'(g)| \leq B_1 L_t + 2CG_t =: M_t.$$

Thus h is M_t-Lipschitz on $[0, G_t]$:

$$|h(g) - h(g')| \le M_t |g - g'| \quad \forall g, g' \in [0, G_t].$$

For any random variable X and any M-Lipschitz function h,

$$\mathrm{Var}(h(X)) \le M^2 \mathrm{Var}(X),$$

which follows from $\mathrm{Var}(Y) = \inf_c \mathbb{E}[(Y - c)^2]$ and Jensen:

$$\begin{aligned}\mathrm{Var}(h(X)) &= \inf_c \mathbb{E}[(h(X) - c)^2] \\ &\le \mathbb{E}[(h(X) - h(\mathbb{E}X))^2] \\ &\le M^2 \mathbb{E}[(X - \mathbb{E}X)^2] \\ &= M^2 \mathrm{Var}(X).\end{aligned}$$

Applying this with $X = \tilde{g}$ and $M = M_t$ gives

$$\mathrm{Var}(F(L_t, \tilde{g})) = \mathrm{Var}(h(\tilde{g})) \le M_t^2 \, \mathrm{Var}(\tilde{g}).$$

Finally, since $\tilde{g}$ is supported on $[0, G_t]$, Popoviciu's inequality yields $\mathrm{Var}(\tilde{g}) \le \frac{(G_t - 0)^2}{4} = \frac{G_t^2}{4}$. Therefore,

$$\mathrm{Var}(F(L_t, \tilde{g})) \le \Big(B_1 L_t + 2CG_t\Big)^2 \frac{G_t^2}{4}.$$

Multiplying by $\alpha^2 B_t$ (or by $(\alpha B_t)^2$ if you keep the looser (16)) gives (17). □

A.2 Proof of Theorem 3.2 (Throughput Optimality / Packing)

Theorem 3.2 (restated). Fix (L_t, G_t) and define the allocated tokens in flight as

$$n(B) := B(L_t + G_t).$$

Assume there exists a function $\phi : [1, T_{\max}] \to \mathbb{R}_{>0}$ such that the step time is

$$T(B) = n(B)\, \phi(n(B)),$$

and $\phi(\cdot)$ is non-increasing on $[1, T_{\max}]$ (seconds per token does not worsen as occupancy increases, up to the memory limit). Define throughput as tokens per second:

$$S(B) := \frac{n(B)}{T(B)}.$$

Under the constraint $n(B) \le T_{\max}$, $S(B)$ is maximized by maximizing $n(B)$, i.e.,

$$B_t^* = \left\lfloor \frac{T_{\max}}{L_t + G_t} \right\rfloor.$$

Proof. By substitution,

$$S(B) = \frac{n(B)}{n(B)\phi(n(B))} = \frac{1}{\phi(n(B))}.$$

Because ϕ is non-increasing, $1/\phi$ is non-decreasing. Hence $S(B)$ is non-decreasing in $n(B)$. Therefore, maximizing $S(B)$ over all feasible B with $n(B) \le T_{\max}$ is equivalent to maximizing $n(B)$ subject to $n(B) \le T_{\max}$, which is achieved by taking the largest integer B such that $B(L_t + G_t) \le T_{\max}$, i.e.,

$$B_t^* = \left\lfloor \frac{T_{\max}}{L_t + G_t} \right\rfloor.$$

□

A.3 Proof of Proposition 3.4 (Dataset Velocity)

Proposition 3.4 (restated with explicit assumptions). Let the dataset contain N prompts and suppose each optimizer iteration consumes exactly B_t distinct prompts (no replay within an epoch), so that the cumulative number of consumed prompts after k iterations is

$$\mathcal{N}(k) := \sum_{t=0}^{k-1} B_t.$$

Define the epoch-completion iteration count as

$$\tau := \min\{k \in \mathbb{N} : \mathcal{N}(k) \ge N\}.$$

Assume the baseline uses a fixed batch size B_{base} for all t, while Flux-RL uses batch sizes $\{B_t^{\text{flux}}\}_{t \ge 0}$ satisfying

$$B_t^{\text{flux}} \ge B_{\text{base}} \quad \forall t, \qquad \text{and} \qquad \exists t_0 \text{ such that } B_{t_0}^{\text{flux}} > B_{\text{base}}.$$

Then the epoch-completion iteration count under Flux-RL is strictly smaller:

$$\tau_{\text{flux}} < \tau_{\text{base}}.$$

Proof. For any k,

$$\mathcal{N}_{\text{flux}}(k) = \sum_{t=0}^{k-1} B_t^{\text{flux}} \ge \sum_{t=0}^{k-1} B_{\text{base}} = k\, B_{\text{base}} = \mathcal{N}_{\text{base}}(k).$$

Moreover, because there exists t_0 with strict inequality, we have strict inequality for all $k > t_0$:

$$\mathcal{N}_{\text{flux}}(k) > \mathcal{N}_{\text{base}}(k) \quad \forall k > t_0.$$

Let τ_{base} be the smallest k such that $\mathcal{N}_{\text{base}}(k) \ge N$. Then

$$\mathcal{N}_{\text{flux}}(\tau_{\text{base}}) \ge \mathcal{N}_{\text{base}}(\tau_{\text{base}}) \ge N,$$

so $\tau_{\text{flux}} \le \tau_{\text{base}}$. If $\tau_{\text{base}} > t_0$, then strict inequality holds:

$$\mathcal{N}_{\text{flux}}(\tau_{\text{base}} - 1) > \mathcal{N}_{\text{base}}(\tau_{\text{base}} - 1) < N,$$

and therefore Flux-RL reaches N strictly earlier, i.e., $\tau_{\text{flux}} < \tau_{\text{base}}$. If $\tau_{\text{base}} \le t_0$, then the strictness condition does not activate within the epoch window and only $\tau_{\text{flux}} \le \tau_{\text{base}}$ is guaranteed. □